

BanLegit-Cite: Execution Plan — Day 1 to ICCIT Submission

Research Management Layer (builds on the Operating Manual as single source of truth)

1. WHERE DO WE START, AND WHY

Start with Stage 2–4 (Novelty Validation → Gap), not Stage 1 in isolation, and not data collection.

Reasoning: your RQ (Stage 1) already exists in draft form from the prior sessions — what doesn't exist yet is *verified* confidence that it survives contact with the literature. Every hour spent on taxonomy design, scraping, or annotation before the novelty claim is stress-tested is an hour you might have to throw away if a competing paper turns up. This is the highest-leverage, lowest-cost stage to do first: it's pure reading and reasoning, requires zero infrastructure, and gates everything else.

Concrete Day 1 action: both researchers spend the first 3 working days *together*, not split, reading the six confirmed competitor papers in full (LePhantomCite, LegalCiteBench, SG-LegalCite, LeCNet, BenHalluEval, Dahl et al. 2024) and jointly producing the novelty table. Do not parallelize this — a shared, argued-over understanding of exactly what's been done is the foundation both of you will build on for four months, and disagreements here are cheap to resolve now and expensive to resolve in week 10.

After that 3-day joint sprint, split (see Section 3).

2. PHASE-BY-PHASE PLAN

Five phases, mapped to the Operating Manual's 23 stages, each ending in a hard go/no-

go gate.

Phase 0 — Foundation (Week 1)

Joint work. Novelty validation, gap statement, RQ/H1-H3 finalization, dataset scope lock, institutional access secured (DLR/BLC/ALR, law-faculty adjudicator identified and confirmed — start this recruitment on Day 1, not Week 5, since it's the slowest-moving external dependency in the whole project). **Gate:** Novelty table + frozen RQ/H + confirmed adjudicator availability, or the project doesn't proceed to Phase 1.

Phase 1 — Design (Weeks 2-3)

Split work begins. Taxonomy drafting, benchmark task/metric design, annotation guideline skeleton, source-repository cataloging, scraper architecture design. **Gate:** Taxonomy + guideline pass pilot round at $\kappa \geq 0.6$.

Phase 2 — Construction (Weeks 4-10)

Heaviest split-parallel phase. Data collection, fabrication generation, full double-annotation, IAA computation, adjudication, gold-set freeze. **Gate:** Gold dataset frozen, IAA ≥ 0.6 overall.

Phase 3 — Experimentation (Weeks 11-13)

Split, converging. Baseline runs (standard + agentic), statistical testing, error analysis. **Gate:** Results matrix complete, H1-H3 answered with significance tests.

Phase 4 — Writing & Submission (Weeks 13-16, overlapping Phase 3)

Joint, writing starts earlier in parallel per the Operating Manual's guidance. Draft, reviewer simulation at all 5 checkpoints, reproducibility pass, open-source release, final submission. **Gate:** Submission checklist (Operating Manual Part 8) fully checked.

3. TWO-RESEARCHER RESPONSIBILITY MATRIX

Split by **track**, not by stage — each researcher owns an end-to-end vertical, which minimizes handoff overhead and duplicated context-switching. Writing and the three joint gates are shared.

Researcher A — Data & Legal Content Lead (own the corpus, taxonomy, annotation, legal correctness) **Researcher B — Engineering & Experiments Lead** (own the pipeline, baselines, stats, infra)

Stage/Task	Owner	Rationale
Novelty validation, gap statement	Joint (Phase 0)	Foundational, must be shared understanding
Dataset scope & source access	A (leads), B (supports scraper feasibility check)	A has the legal-content judgment; B knows what's technically scrapable
Citation taxonomy design	A	Requires legal-domain judgment throughout
Benchmark task/metric design	B	Engineering/statistics-facing
Scraper + OCR pipeline	B	Pure engineering
Annotation guideline writing	A	Legal-content judgment
Recruiting/managing annotators + adjudicator	A	Requires legal-community relationships
Fabrication generation (prompt design, batch review)	A (content review) + B (pipeline/automation)	Split cleanly: B builds the batch-and-gate system, A approves/rejects batch content
IAA computation, adjudication logging	A (adjudication content) + B (statistical computation)	

Stage/Task	Owner	Rationale
Benchmark packaging/splits	B	Engineering
Baseline model runs (standard + agentic)	B	Engineering/infra-heavy
Statistical significance testing	B	
Error analysis (content interpretation)	A (legal-error interpretation) + B (technical failure patterns)	Both needed — errors have both a "why did the model fail linguistically/legally" angle (A) and a "why did the model fail architecturally" angle (B)
Reproducibility pass	B	
Open-source release / copyright check	A (copyright/licensing judgment) + B (technical release)	
Paper writing	Joint , section-split: A writes Intro/Related Work/Taxonomy/Annotation sections; B writes Methods(Experiments)/Results/Reproducibility sections	Each writes what they built — avoids one person becoming a bottleneck translator of the other's work
Reviewer simulation response	Joint	Both must internalize every flag

Stage/Task	Owner	Rationale
Final submission decision	Joint, PI has final call	Per Operating Manual Part 0

4. JOINT VS. INDEPENDENT TASKS — EXPLICIT RULE

Always joint (never split):

- Novelty/gap validation (Phase 0)
- Any taxonomy category boundary decision
- Any change to the frozen RQ/hypotheses (requires both signatures in the decision log)
- Adjudication of high-disagreement annotation cases
- The five Reviewer Simulation checkpoints
- Final submission go/no-go

Always independent (parallel, no blocking):

- Scraper/pipeline engineering (B) while annotator recruitment happens (A)
- Baseline model runs (B) while error-analysis-relevant legal review happens (A) on a subset of already-frozen results
- Section-drafting once the underlying stage is complete (each writes their own section without waiting on the other)

Sequential but single-owner (not joint, but blocks the other's downstream work — plan for it):

- Taxonomy freeze (A) blocks fabrication generation (A+B) and guideline finalization

- Gold-set freeze (A+B jointly at IAA stage) blocks all of Phase 3
 - This is why Phase 0's 3-day joint sprint matters — get the blocking joint work done fast so the long independent stretches (Phase 2's 6-7 weeks) can actually run in parallel without one person idling.
-

5. AI AGENTS VS. HUMAN SUPERVISION — OPERATIONAL SUMMARY

This restates the Operating Manual's Part 0 matrix in execution terms — who *triggers* each agent and who *reviews* its output:

Task	AI Agent Does	Human Does
Literature search & metadata extraction	A2 finds papers, extracts structured fields	Researcher (either) spot-checks 10%, reads top competitors in full
Novelty adversarial search	A3 searches for counter-evidence	PI/both researchers read flagged papers, make the judgment
Taxonomy drafting	B2 proposes categories/examples	Researcher A finalizes boundaries, both pilot-test with real annotators
Scraper code	Coding agent (Section 6) writes/debugs scraper	Researcher B reviews diffs, runs spot-checks against source
Fabrication generation	C2 generates batches	Researcher A reviews $\geq 20\%$ of each batch before next batch unlocks
Citation cross-checking during annotation	C4 surfaces evidence	Human annotator makes the actual real/fake call
IAA computation	C5 (or plain code) computes κ	Researcher B verifies computation against raw data; adjudicator (human legal expert) resolves disagreements
Baseline execution	D2 runs models via coding agent	Researcher B approves prompts beforehand, reviews for crashed/implausible runs
Statistical tests	D3 runs pre-registered tests	Researcher B confirms test matches pre-registration
Draft writing	E1 drafts sections from artifacts	Both researchers read full draft against artifact trail

Task	AI Agent Does	Human Does
Pre-submission review	E2 simulates reviewer	PI resolves every fatal-flaw flag personally

Rule of thumb: agents do anything that is *generation, extraction, or computation at scale*; humans do anything that is *judgment about correctness, novelty, or legal fact*. This line should never blur, especially for gold-label legal ground truth.

6. AGENTIC CODING TOOLS — CONCRETE STACK, NOT A MENU

Decision: standardize on Claude Code as the primary agentic coding tool for the whole project, with Cursor as the IDE for interactive/manual coding sessions. Do not run Antigravity, Gemini CLI, and Codex in parallel — tool sprawl creates inconsistent conventions across a two-person team and adds config/context overhead with no offsetting benefit at this project's scale.

Tool	Role	Why this and not an alternative
Claude Code	Primary agent for: scraper/pipeline construction, fabrication-generation batch scripts, evaluation harness (baseline runner across models/settings), statistical analysis scripts, repo scaffolding, and long-running multi-file refactors (e.g., restructuring the data schema across raw/annotated/gold)	Handles long-context, multi-file repo reasoning well; can be given the Operating Manual itself as a project-level instruction file so every agent-generated script follows the batch-and-gate, provenance-metadata, and evidence-logging conventions automatically, rather than each researcher re-explaining constraints per session
Cursor	Day-to-day IDE for Researcher B during hands-on debugging, quick fixes, and reviewing Claude Code's diffs before merging	Best for fast, low-latency interactive editing where a human wants to stay in the loop line-by-line, complementary to Claude Code's larger autonomous tasks rather than competing with it
Gemini CLI	Optional, narrow use: bulk OCR/summarization pass over large scanned judgment PDFs if volume is high, since long-context PDF handling is its strength	Only invoke if Stage 8's scanned-PDF volume becomes a bottleneck; not part of the default stack
OpenAI Codex / Antigravity	Not adopted	No task in this project needs a second agentic coding tool; adding one only fragments config/prompt-convention consistency across the team for zero measurable benefit at two-person scale

Concrete setup on Day 1: create a `CLAUDE.md` (or equivalent project-instructions file) at the repo root containing the Operating Manual's evidence rules, provenance-metadata requirements, and batch-and-gate constraints, so every Claude Code session in this repo inherits these constraints automatically instead of relying on each researcher to restate them per session.

7 & 8. RESEARCH TOOL STACK — ONE TOOL PER STAGE, WITH JUSTIFICATION

Concrete assignment, not a menu of options:

Stage / Need	Tool	Why this tool specifically
Initial competitor discovery (Phase 0)	Connected Papers	Visual citation-graph exploration from your 6 known seed papers (LePhantomCite, LegalCiteBench, etc.) surfaces adjacent/cited-by work fastest — best for the 3-day joint sprint
Bulk structured metadata extraction (powers A2 agent)	Semantic Scholar API + OpenAlex API	Both are free, programmatic, and feed directly into the literature database schema (Section 6.5 of the Operating Manual) — use Semantic Scholar for citation counts/influence, OpenAlex for broader bulk coverage and open metadata, called by Claude Code scripts rather than browsed manually
Continuous novelty monitoring (recurring monthly check, Stage 2)	Litmaps	Purpose-built for ongoing citation-alert tracking from a seed set — set this up once in Week 1 pointed at your 6 core papers, and it will flag new citing/related work automatically through submission, which is exactly the recurring competitor-check the Operating Manual requires
Adversarial novelty checking / claim verification (A3 agent support)	Consensus	Designed specifically to answer yes/no scientific-claim questions grounded in papers — well-suited to A3's job of actively trying to falsify "no prior work does X"
Citation-context sentiment (does a paper support or contradict a claim?)	Scite	Shows whether a paper is cited approvingly, contrastingly, or just mentioned — useful when checking whether e.g. an agentic-verification paper's finding is contested elsewhere before you rely on it (Stage 18's H3 design leans on LePhantomCite's agentic finding — worth Scite-

Stage / Need	Tool	Why this tool specifically
		checking whether that finding has been challenged)
Deep synthesis across your core paper set, grounded Q&A	NotebookLM	Upload the 6 competitor papers + your own taxonomy/guideline drafts; ask grounded questions ("does LePhantomCite's taxonomy handle statutory citations at all?") with answers citable back to exact source passages — lowest-hallucination-risk tool for this specific need, and useful for onboarding new annotators with a generated audio/text overview
Structured data extraction from papers at scale (e.g., pulling every competitor's dataset size/metric into your comparison table)	Elicit	Purpose-built for extracting structured fields (sample size, method, outcome) across many papers into a table — faster than manual extraction for A2's spreadsheet-building task
Exploratory citation-network discovery beyond your seed set	ResearchRabbit	Complements Connected Papers with a different discovery algorithm (author-network + co-citation) — run once during Phase 0 to catch anything Connected Papers' graph missed, not a recurring tool
Quick fact-checks / fast lookups during writing	Perplexity	Fine for quick, low-stakes lookups (e.g., "what's the current IEEE citation format for X"), but never used as evidence for a novelty or comparative claim — anything Perplexity surfaces that matters for the paper must be independently verified

Stage / Need	Tool	Why this tool specifically
		against the primary source before use, since it does not meet this project's evidence-logging bar
Citation counts / venue legitimacy checks	Google Scholar	Standard for checking whether a paper is peer-reviewed, its citation count, and venue — quick sanity-check tool, not a primary research tool
Reference management, single source of truth for bibliography	Zotero	Central repository all researchers and the E1 writing agent pull from; browser plugin captures PDFs directly; syncs with the paper's citation manager (BibTeX export) — this is the canonical bibliography, Litmaps/Consensus/Elicit findings all get funneled into it, not treated as separate parallel bibliographies

Anti-pattern to avoid: don't use five tools to do the same job "just in case." Each tool above has exactly one job in this stack. If you find yourself cross-checking the same competitor-search question in three tools out of habit, that's wasted time — Connected Papers + Litmaps + Consensus already cover discovery, monitoring, and claim-verification respectively; that's sufficient.

9. HUMAN-IN-THE-LOOP WORKFLOW FOR TWO RESEARCHERS

Three tiers of review, matched to the Operating Manual's decision matrix:

Tier 1 — Async, individual review (daily): each researcher reviews their own track's agent outputs before committing (e.g., B reviews Claude Code's scraper diffs; A reviews C2's fabrication batches). No meeting required.

Tier 2 — Paired review (weekly, or at any Phase gate): both researchers jointly review anything that crosses tracks — taxonomy edge cases surfaced during fabrication

generation, error-analysis findings that need both legal and technical interpretation, draft sections referencing the other's work.

Tier 3 — Formal gate review (5 fixed checkpoints, per Operating Manual Part 5): both researchers plus, where relevant, the legal adjudicator, formally sign off before the project proceeds — these are the only points where a Reviewer Simulation Agent run is mandatory and where the decision log gets a dated joint entry.

Rule: nothing skips from Tier 1 straight to submission. Anything touching the frozen RQ, taxonomy, or gold labels must pass through Tier 2 at minimum.

10. WORKFLOW TO MINIMIZE DUPLICATION AND MAXIMIZE EVIDENCE QUALITY

Four concrete mechanisms, not general advice:

1. **Single bibliography, single source of truth (Zotero).** Every tool in Section 7 feeds into it; no researcher maintains a separate reading list. Prevents the classic two-person failure mode of both independently re-discovering the same paper weeks apart.
2. **CLAUDE.md project-instructions file** (Section 6) means every agent invocation, by either researcher, inherits the same evidence-logging and batch-and-gate rules automatically — prevents Researcher A's agent sessions from following looser conventions than Researcher B's.
3. **Decision log is append-only and shared,** reviewed by both at the start of each week — a 10-minute Monday read prevents the two tracks from silently drifting on assumptions (e.g., B assuming a taxonomy category was finalized when A hasn't actually pilot-tested it yet).
4. **RESULTS.md is updated the same day any result is produced,** not batched at the end — this is the single highest-leverage reproducibility habit, since reconstructing

"which config produced this number" three months later is far more expensive than logging it in real time.

RECOMMENDED SOFTWARE STACK

Layer	Tool	Notes
Version control	Git + GitHub	Private repo until release-ready
Large file / dataset versioning	DVC (Data Version Control)	Git tracks code/config; DVC tracks the actual corpus files, pointing to a remote (e.g., Google Drive or S3) — prevents bloating the git repo with binary/large text corpora
Annotation platform	Label Studio	Open-source, supports custom taxonomy schemas, multi-annotator agreement tracking natively
Experiment tracking	Weights & Biases (free tier)	Log every baseline run (model × setting × category) with config, cost, and output artifact links — gives both researchers a shared dashboard instead of scattered result files
Coding agent	Claude Code (primary) + Cursor (IDE)	Per Section 6
Statistical computation	Python: <code>scipy.stats</code> , <code>statsmodels</code> , <code>scikit-learn</code>	Standard, reproducible, well-documented

Layer	Tool	Notes
Retrieval index (agentic baseline)	FAISS or lightweight Elasticsearch	Over the Stage 8 source corpus for the agentic-setting search tool
Reference management	Zotero + BibTeX export	Feeds directly into the LaTeX paper
Paper writing	Overleaf (shared LaTeX)	Real-time joint editing, matches ICCIT LaTeX template requirements
Communication/async log	Notion or a plain shared markdown repo folder	Research log + decision log live here, git-tracked for permanence

RESEARCH TOOL STACK (SUMMARY TABLE)

Purpose	Primary Tool	Secondary/Backup
Seed-paper discovery	Connected Papers	ResearchRabbit
Bulk metadata	Semantic Scholar + OpenAlex API	—
Ongoing monitoring	Litmaps	Manual monthly Google Scholar check
Claim verification	Consensus	Scite
Deep synthesis/grounded Q&A	NotebookLM	—
Structured extraction	Elicit	Manual extraction as fallback

Purpose	Primary Tool	Secondary/Backup
Quick lookups (non-evidentiary)	Perplexity	Google Scholar
Reference management	Zotero	—

DAILY WORKING STRATEGY

- **Morning (both, 15 min async):** each posts a 3-line update in the research log — what shipped yesterday, what's blocked, what's planned today. No live meeting needed unless a blocker crosses tracks.
- **Core hours (independent, per track):** A works legal-content/annotation track; B works engineering/experiments track; each drives their own agent sessions per Section 5/6.
- **End of day (both, 10 min):** commit + push; update RESULTS.md or research log if anything result-bearing was produced that day — never let this accumulate to "catch up later."
- **Weekly (both, 45-60 min, fixed slot):** Tier 2 paired review (Section 9); review decision log; confirm next week's plan against the Section 2 phase gates.
- **At each of the 5 formal gates:** half-day joint session — run Reviewer Simulation, resolve flags, sign the decision log entry together before proceeding.

COLLABORATION STRATEGY

- Track ownership (Section 3) is fixed for the whole project — do not reshuffle mid-phase, since context-switching cost between the legal-content and engineering tracks is high.

- Disagreements about taxonomy or gap framing are resolved in the weekly Tier-2 session, not over async chat — these decisions are expensive to get wrong and benefit from real-time back-and-forth.
 - Both researchers read the full paper draft before every Reviewer Simulation run, not just their own sections — this is non-negotiable per the Operating Manual's "PI reads full draft" rule, and with two researchers, both count as PI-level reviewers for this purpose.
-

GIT WORKFLOW

- **Branching:** trunk-based with short-lived feature branches, named by stage ID from the Operating Manual (e.g., `stage7-taxonomy`, `stage13-baselines`).
 - **Commits:** every commit message references the stage and, where applicable, the agent that produced the underlying draft (e.g., `[stage8b] fabrication batch 12, C2-generated, A-reviewed, 3 rejected`).
 - **PRs:** every merge into `main` requires the *other* researcher's review, even for solo-track work — cheap cross-track visibility that catches drift early.
 - **Protected main:** `main` only receives merges that pass this review; no direct pushes.
 - **Data:** never commit raw corpus/annotation files directly to git — DVC-tracked, git stores only the DVC pointer files.
 - **Tags:** tag the exact commit used to produce each paper table/figure (`results-table3-v1`), referenced in RESULTS.md — this is what makes Stage 19 reproducibility checkable by commit hash, not just by description.
-

LITERATURE MANAGEMENT WORKFLOW

1. Discovery (Connected Papers/ResearchRabbit) → candidate paper.

2. Add to Zotero immediately with full metadata + PDF.
 3. A2 agent extracts structured fields into `literature/bibliography.csv` (Operating Manual Section 6.5 schema).
 4. If plausibly competing: routed to A3 for adversarial novelty check, logged in `literature/novelty_report.md`.
 5. Litmaps monitors the paper going forward for citations/related work.
 6. Monthly: both researchers review the Litmaps digest together (10 min), decide if anything needs a full read.
-

EXPERIMENT MANAGEMENT WORKFLOW

1. D1 drafts protocol (prompts, temp, seeds) → PI (both) approves before any run.
 2. Every run logged to W&B with full config.
 3. D2 executes via Claude Code-orchestrated scripts, writing raw outputs to `experiments/results/` with timestamp + config hash in filename.
 4. Failed/implausible runs flagged, never silently excluded — logged in the run index with a reason.
 5. D3 runs pre-registered stats only, output linked back to the exact run IDs used.
 6. RESULTS.md updated same day, mapping paper table → script → config → run ID.
-

BEST PRACTICES FROM TOP AI RESEARCH LABS, APPLIED HERE

- **Pre-registration culture (DeepMind/Anthropic-style eval hygiene):** hypotheses and metrics locked (Stage 5) before data collection — already built into this plan; resist any temptation to add a metric after seeing favorable results.
- **"Someone else can reproduce it" as the actual bar**, not "we described the method in prose" — enforced via the reproducibility dry-run in Stage 19, done by whichever

researcher didn't build the pipeline.

- **Red-team your own claims** — the Novelty Verification Agent (A3) and Reviewer Simulation Agent (E2) exist specifically to play this adversarial role continuously, not just before submission, mirroring how frontier labs run internal red-teams throughout a project rather than only at launch.
- **Living documentation over end-of-project write-up** — RESULTS.md, decision log, and research log updated same-day, not reconstructed from memory in week 15.
- **Dataset versioning as seriously as code versioning** (Section 6.6 of the Operating Manual) — frontier labs treat eval sets as artifacts with their own release discipline, not incidental files.
- **Narrow, falsifiable claims over broad ones** — the entire novelty positioning (jurisdictional + linguistic gap, not methodological) mirrors how strong papers scope their contribution tightly enough to defend it completely, rather than overclaiming and leaving it exposed.